I5 · Conditions et boucles

~2.5 h

À la fin de cette séance vous saurez

- exécuter du code seulement si une condition est vraie avec `if` / `else if` / `else` ;
- classer une colonne entière avec `ifelse()` et `dplyr::case_when()` ;
- répéter une tâche avec une boucle `for`, stocker les résultats et éviter les erreurs courantes ;
- savoir quand une boucle n'est **pas** nécessaire (code vectorisé, `tapply()`, `group_by()`).

i Cette séance utilise le tableau d'analyse de la séance I3 :
`seaweed <- read.csv("data/checkpoints/I3_seaweed_analysis.csv")`.

⇩ checkpoint: I3_seaweed_analysis.csv

Pourquoi automatiser ?

La coopérative veut, **pour chacun des 12 villages**, un petit résumé et un fichier CSV à envoyer au représentant du village. Copier les mêmes 6 lignes de code 12 fois est lent et dangereux : un changement oublié et un village reçoit de faux chiffres. Les conditions et les boucles permettent d'écrire la logique **une seule fois**.

if, else if, else

`if (condition) { ... }` exécute le bloc seulement si la condition vaut **un seul** `TRUE`. Ajoutez `else if` et `else` pour les autres cas.

```
loss_rate <- mean(seaweed$status != "harvested") * 100
loss_rate
```

```
#> [1] 10.01942
```

```
if (loss_rate > 15) {
  message("Alert: many lines lost")
} else if (loss_rate > 5) {
  message("Losses to watch: ", round(loss_rate, 1), " %")
} else {
  message("Losses are low")
}
```

```
#> Losses to watch: 10 %
```

! `if` demande **un seul** TRUE/FALSE. `if (seaweed$sgr > 4)` est une erreur (« the condition has length > 1 »), car il y a 2 575 valeurs. Pour classer une colonne entière, utilisez `ifelse()` ou `case_when()` (section suivante). De plus, `if (NA)` est une erreur : testez d'abord les valeurs manquantes avec `is.na()`.

Classer une colonne : ifelse() et case_when()

`ifelse(test, oui, non)` fonctionne sur un vecteur entier. Les `ifelse()` imbriqués deviennent illisibles au-delà de 2–3 classes : `case_when()` (dplyr) liste les cas un par ligne, `condition ~ valeur`, testés dans l'ordre ; `TRUE ~` attrape tout le reste.

IFELSE()

```
seaweed$growth_class <- ifelse(is.na(seaweed$s
gr), "lost",
                        ifelse(seaweed$sgr >=
4, "good",
                        ifelse(seaweed$sgr >=
3, "medium", "poor")))
table(seaweed$growth_class)
```

```
#>
#>   good   lost medium   poor
#>    473    258   1229    615
```

CASE_WHEN()

```
library(dplyr)
seaweed <- seaweed %>% mutate(growth_class = c
ase_when(
  is.na(sgr) ~ "lost",
  sgr >= 4   ~ "good",
  sgr >= 3   ~ "medium",
  TRUE       ~ "poor"))
table(seaweed$growth_class)
```

```
#>
#>   good   lost medium   poor
#>    473    258   1229    615
```

`cut()` est une autre option pour des classes numériques à limites fixes (`right = FALSE` place la limite dans la classe supérieure) :

```
table(cut(seaweed$sgr, breaks = c(0, 3, 4, Inf), labels = c("poor", "medium", "good"), right = FAL
SE))
```

```
#>
#>   poor medium   good
#>    615   1229    473
```

Boucles for

`for (v in valeurs) { ... }` exécute le bloc une fois pour chaque valeur, rangée dans `v`. Trois schémas couvrent la plupart des besoins.

1. Afficher ou enregistrer quelque chose pour chaque groupe

```
villages <- sort(unique(seaweed$village))
for (v in villages[1:3]) {
  d <- seaweed[seaweed$village == v, ]
  cat(v, ": ", nrow(d), " lines, mean SGR = ", round(mean(d$sgr, na.rm = TRUE), 2), " %/day\n", se
p = "")
}
```

```
#> Ambalakely: 147 lines, mean SGR = 3.34 %/day
#> Ambatofotsy: 199 lines, mean SGR = 3.43 %/day
#> Ampasimena: 240 lines, mean SGR = 3.42 %/day
```

```
dir.create("outputs/villages", showWarnings = FALSE)
for (v in villages) {
  d <- seaweed[seaweed$village == v, ]
  write.csv(d, file.path("outputs/villages", paste0("lines_", v, ".csv")), row.names = FALSE)
}
```

2. Remplir un vecteur de résultats

Créez un contenant vide **avant** la boucle, de la bonne longueur, puis remplissez un élément par tour. `seq_along(x)` donne 1, 2, …, length(x) — plus sûr que `1:length(x)` quand `x` est vide.

```
loss <- numeric(length(villages))          # 12 zeros
names(loss) <- villages
for (i in seq_along(villages)) {
  d <- seaweed[seaweed$village == villages[i], ]
  loss[i] <- round(100 * mean(d$status != "harvested"), 1)
}
loss
```

```
#>  Ambalakely Ambatofotsy  Ampasimena   Andrevobe    Ankarabe   Antsiraka
#>         8.2        11.1        12.1         7.7         4.5        10.4
#>      Befasy      Lovobe    Mahavelo  Sarondrano Tsiandavaka  Tsimanjaka
#>        14.2        13.6         8.6         8.9        13.3         8.8
```

3. Construire un tableau, une ligne par tour

```
res <- vector("list", length(villages))     # an empty list
for (i in seq_along(villages)) {
  d <- seaweed[seaweed$village == villages[i], ]
  res[[i]] <- data.frame(village = villages[i], n_lines = nrow(d),
                         mean_sgr = mean(d$sgr, na.rm = TRUE),
                         harvest_t = sum(d$final_weight_kg, na.rm = TRUE) / 1000)
}
summary_v <- do.call(rbind, res)            # stack the 12 one-row tables
head(summary_v, 4)
```

```
#>       village n_lines mean_sgr harvest_t
#> 1  Ambalakely     147 3.342884    6.9973
#> 2 Ambatofotsy     199 3.434218    9.7298
#> 3  Ampasimena     240 3.415613   11.3207
#> 4   Andrevobe     182 3.461735    9.2724
```

> **i** **Avez-vous vraiment besoin d'une boucle ?** Beaucoup de boucles peuvent être remplacées par du code vectorisé ou des résumés par groupe, plus courts et plus rapides :

```
round(100 * tapply(seaweed$status != "harvested", seaweed$village, mean), 1)   # = loss
```

```
#>  Ambalakely Ambatofotsy  Ampasimena   Andrevobe    Ankarabe   Antsiraka
#>         8.2        11.1        12.1         7.7         4.5        10.4
#>      Befasy      Lovobe    Mahavelo  Sarondrano Tsiandavaka  Tsimanjaka
#>        14.2        13.6         8.6         8.9        13.3         8.8
```

Utilisez une boucle quand chaque tour fait quelque chose sans équivalent vectorisé : **écrire des fichiers**, tracer **un graphique par groupe**, lire **plusieurs fichiers**, lancer une étape longue en affichant la progression.

Lire plusieurs fichiers avec une boucle

```
files <- list.files("outputs/villages", pattern = "\\.csv$", full.names = TRUE)
all <- vector("list", length(files))
for (i in seq_along(files)) all[[i]] <- read.csv(files[i])
all <- do.call(rbind, all)                  # one table again
```

› **À vous.** Écrivez une boucle sur les trois zones qui affiche la zone et le nombre de lignes **perdues lors d'une tempête**.

```
for (z in unique(seaweed$zone)) { cat(z, sum(seaweed$zone == z & seaweed$status == "lost - storm"), "\n") }
```

while et break (rarement utiles)

`while (condition) { ... }` répète tant que la condition est vraie — par exemple jusqu'à atteindre un objectif. `break` sort d'une boucle plus tôt ; `next` passe au tour suivant. Assurez-vous que la condition devient `FALSE`, sinon la boucle ne s'arrête jamais (appuyez sur **Échap** dans RStudio).

```
# How many days for 10 kg of cuttings to reach 30 kg at 3.5 % per day?
w <- 10; day <- 0
while (w < 30) { w <- w * exp(0.035); day <- day + 1 }
day
```

```
#> [1] 32
```

Travail d'application et quiz

Travail d'application I5

Script `scripts/i05_loops.R`, avec le tableau d'analyse I3.

1. Créez `growth_class` avec `case_when()` : `lost` (pas de SGR), `good` (≥ 4), `medium` (≥ 3), `poor` (sinon). Combien de lignes **good** ? Combien de **poor** ?
2. Avec une boucle `for` sur les villages, calculez le % de lignes non récoltées (perdues). Quel village a le taux de perte le plus élevé, et combien (1 décimale) ?
3. Avec une boucle, construisez le tableau de synthèse (une ligne par village) avec la récolte totale en **tonnes**. Récolte totale de **Mahavelo** (3 décimales) ? Quel village a le plus récolté ?
4. Combien de lignes ont été perdues lors d'une tempête dans la zone **South** ?
5. Avec `while` : combien de jours pour que 12 kg de boutures dépassent 50 kg à 4 % par jour (`w * exp(0.04)` chaque jour) ?
6. Combien de lignes ont été semées pendant la **saison des pluies 2024** ?

by Aroo Faliniriana

7. Utilisez la boucle pour écrire un CSV par village dans `outputs/villages/` (non noté, mais utile pour la séance d'exercices).

Quiz : répondez en ligne sur le site du cours (votre score y est enregistré).

by Aroo Falinirina